

Informe de Desarrollo del Proyecto UdeAStay

Daniel Rúa, Tomás Mesa
Facultad de Ingeniería
Departamento de Ingeniería Electrónica y Telecomunicaciones
Universidad de Antioquia



Resumen—Este documento detalla el análisis, diseño, implementación y evolución de la solución para el proyecto UdeAStay, un sistema de administración para un mercado de estadias hogareñas. El desarrollo se adhiere a los principios de la Programación Orientada a Objetos en C++, cumpliendo con restricciones específicas como la no utilización de la STL y el manejo explícito de memoria dinámica. Se presenta el análisis del problema, la descripción del diseño de clases resultante y sus relaciones, la lógica de subprogramas clave, los desafíos afrontados y las consideraciones para futuras implementaciones, de acuerdo con los requisitos del Desafío II de Informática II.

Index Terms—Programación Orientada a Objetos, C++, Diseño de Software, Persistencia de Datos, Gestión de Memoria Dinámica, UdeAStay.

1. INTRODUCCIÓN

El proyecto UdeAStay surge como respuesta al desafío de modelar y administrar un mercado de estadias hogareñas, una modalidad de alojamiento que fomenta la inmersión cultural. El objetivo principal de este desarrollo es crear un sistema robusto y eficiente en C++ que permita la gestión de alojamientos, anfitriones, huéspedes y reservaciones, utilizando los fundamentos de la Programación Orientada a Objetos (POO). Este sistema no solo debe cumplir con las funcionalidades especificadas, como la carga y guardado de datos, el inicio de sesión por perfiles, la reserva y anulación de alojamientos, la gestión de históricos y la medición de recursos, sino también adherirse a restricciones importantes como la no utilización de la Standard Template Library (STL) y la implementación de estructuras de datos propias con manejo de memoria dinámica. Este informe documenta el proceso de desarrollo, desde el análisis inicial hasta las consideraciones finales, sirviendo como evidencia del trabajo realizado.

2. ANÁLISIS DEL PROBLEMA Y CONSIDERACIONES PARA LA SOLUCIÓN PROPUESTA

El problema central consiste en desarrollar un sistema de gestión para UdeAStay, una plataforma de alquiler de estadias hogareñas. El sistema debe manejar cuatro entidades principales: **Anfitriones** (quienes ofrecen alojamientos), **Huéspedes** (quienes reservan alojamientos), **Alojamientos** (las propiedades en alquiler) y **Reservaciones** (el vínculo transaccional entre un huésped y un alojamiento).

2.1. Requisitos Clave

Los requisitos extraídos del enunciado del proyecto son extensos y detallados. Estos incluyen la gestión de información para cada entidad (alojamientos con detalles como código, nombre, anfitrión, ubicación, tipo, precio, amenidades, y fechas reservadas; anfitriones y huéspedes con datos personales, antigüedad y puntuación), así como la implementación de funcionalidades esenciales. Entre estas funcionalidades se encuentran la carga y actualización de datos desde archivos de texto, un sistema de inicio de sesión diferenciado por rol, la capacidad para huéspedes de buscar y reservar alojamientos (con filtros opcionales), y la opción para anfitriones y huéspedes de anular reservaciones. Adicionalmente, los anfitriones deben poder consultar sus reservaciones activas y el sistema debe permitir la actualización de un histórico de reservaciones. Se deben realizar validaciones importantes, como asegurar que un huésped no tenga reservaciones solapadas y que un alojamiento esté disponible para la totalidad de la reserva. Finalmente, el sistema debe reportar automáticamente la cantidad de iteraciones y una estimación de la memoria consumida al finalizar cada funcionalidad, todo bajo la restricción de no usar la STL y de implementar estructuras de datos propias con memoria dinámica en C++ bajo el paradigma POO.

2.2. Alternativa de Solución Propuesta

La solución se basa en un diseño orientado a objetos, modelando cada entidad principal del problema como una clase. Se definieron las clases principales 'Anfitrión', 'Huesped', 'Alojamiento', 'Reserva', y una clase orquestadora 'SistemaUdeAStay'. Se descartó la herencia para los roles de usuario para enfocarse en la composición. Se crearon clases auxiliares como 'ListaPersonalizadaTz' (para listas dinámicas), 'Fecha' (para manejo de fechas), y 'Credenciales' (para datos de login). La persistencia de datos se maneja mediante archivos de texto con un formato de delimitadores diseñado específicamente, y 'SistemaUdeAStay' centraliza estas operaciones. La gestión de memoria es manual, utilizando 'new' y 'delete', y las clases que manejan recursos dinámicos implementan la Regla de Tres/Cinco. 'SistemaUdeAStay' actúa como dueño de las instancias principales de las entidades. Esta aproximación busca cumplir con todos los requisitos, promoviendo la encapsulación y la modularidad.

3. DIAGRAMA DE CLASES DE LA SOLUCIÓN PLANTEADA

El diseño de la solución se plasma en un diagrama de clases que visualiza las entidades, sus atributos, métodos y las relaciones entre ellas. Este diagrama es fundamental para entender la arquitectura del sistema y cómo sus componentes interactúan.

3.1. Descripción General del Diagrama

El diagrama está compuesto por las siguientes clases principales: 'SistemaUdeAStay', 'Anfitrión', 'Huesped', 'Alojamiento', y 'Reserva'. Adicionalmente, se utilizan clases auxiliares como 'ListaPGenerica' (representación de la plantilla 'ListaPersonalizadaTz'), 'Fecha', y 'Credenciales'. Estas clases auxiliares encapsulan funcionalidades específicas y son utilizadas por las clases principales.

3.2. Justificación de Clases y Relaciones

- **Entidades Principales:** 'Anfitrión', 'Huesped', 'Alojamiento', 'Reserva' modelan los actores y objetos centrales del dominio de UdeAStay. Los atributos comunes a 'Anfitrión' y 'Huesped' (documento, nombre, credenciales, antigüedad, puntuación) se definen en cada una debido a la decisión de no utilizar herencia para estos roles.
- **Clase Orquestadora ('SistemaUdeAStay'):** Centraliza la lógica de la aplicación, gestionando las listas maestras de las entidades principales (manejando su ciclo de vida), la interacción con el usuario (menús), y la persistencia de datos.
- **Clases de Utilidad:**
 - 'ListaPGenerica': Provee la funcionalidad de una lista dinámica, esencial

dada la restricción de no usar STL. Es utilizada por las clases principales para gestionar colecciones.

- 'Fecha': Encapsula la lógica de manejo y formateo de fechas.
- 'Credenciales': Agrupa los datos de autenticación.
- **Relaciones: Es parte de (Composición):** Se utiliza esta relación (rombo relleno en UML) cuando una clase "Todo" contiene y es responsable del ciclo de vida de una clase "Parte". Por ejemplo, 'SistemaUdeAStay' está compuesto por sus listas de entidades; 'Anfitrión' contiene su objeto 'Credenciales' y su lista de alojamientos.
- **Asociaciones Directas:** Representan relaciones donde una clase tiene una referencia a otra, sin implicar propiedad total sobre el ciclo de vida del objeto referenciado. Por ejemplo, un 'Alojamiento' está asociado a un 'Anfitrión' responsable; una 'Reserva' está asociada a un 'Alojamiento' y a un 'Huesped'. La cardinalidad se especifica para indicar cuántas instancias pueden estar involucradas.

4. DESCRIPCIÓN EN ALTO NIVEL DE TAREAS NO TRIVIALES

En esta sección se describe la lógica general de los algoritmos más complejos o fundamentales del sistema.

4.1. Algoritmo de Carga y Guardado de Datos ('SistemaUdeAStay')

La persistencia de datos se realiza a través de archivos de texto con un formato predefinido.

- **Carga de Datos:** Se abren los archivos designados y se leen línea por línea. Cada línea se parsea usando delimitadores para extraer los campos, los cuales se convierten a los tipos de datos apropiados. Se crean nuevas instancias de objetos con estos datos, se almacenan en listas personalizadas dentro de 'SistemaUdeAStay', y se establecen las relaciones entre ellos (ej. enlazar un 'Alojamiento' con su 'Anfitrión').
- **Guardado de Datos:** Se abren los archivos en modo escritura. Se itera sobre las listas de objetos en 'SistemaUdeAStay'. Para cada objeto, sus atributos se formatean en una cadena de texto según el diseño de archivo y se escribe como una línea en el archivo respectivo.

4.2. Algoritmo para Reservar Alojamiento ('SistemaUdeAStay::realizarProcesoReserva')

Esta funcionalidad para huéspedes implica:

1. Recopilar criterios de búsqueda (fecha, municipio, noches, filtros).
2. Iterar la lista de alojamientos del sistema, filtrando por municipio y criterios opcionales. Para cada candidato, se invoca `'Alojamiento::estaDisponible()'` para verificar la no superposición con reservas existentes, lo que requiere una correcta aritmética de fechas.
3. Presentar los alojamientos disponibles al huésped para su selección.
4. Validar que el huésped no tenga reservas solapadas con las nuevas fechas.
5. Si todo es válido, se crea un nuevo objeto `'Reserva'`, se almacena en la lista de reservas activas de `'SistemaUdeAStay'`, y se establecen las referencias en el `'Huesped'` y el `'Alojamiento'`.
6. Se muestra un comprobante de confirmación.

4.3. Algoritmo para Actualizar Histórico de Reservas (`'SistemaUdeAStay::realizarActualizacionHistorico'`)

Iniciada por un anfitrión, esta función gestiona el ciclo de vida de las reservaciones:

1. Se solicita al anfitrión una "fecha de corte".
2. El sistema itera sobre su lista de reservaciones activas.
3. Para cada reservación, si su fecha de finalización es anterior a la "fecha de corte", se considera pasada.
4. Las reservaciones pasadas se mueven de la lista de activas a la lista de históricas (ambas gestionadas por `'SistemaUdeAStay'`).
5. Se actualizan las referencias en los `'Alojamiento'` para reflejar que la reserva ya no está activa.

5. DISEÑO DE ARCHIVOS DE PERSISTENCIA DE DATOS

La persistencia de los datos del sistema UdeAStay se gestiona mediante un conjunto de archivos de texto plano. Cada tipo de entidad principal (Anfitriones, Huéspedes, Alojamientos, Reservaciones Vigentes y Reservaciones Históricas) se almacena en un archivo dedicado (por ejemplo, `anfitriones.txt`, `alojamientos.txt`, etc.).

Dentro de cada archivo, cada línea representa una instancia de un objeto. Los diferentes atributos o campos de cada objeto se separan utilizando un delimitador principal, que es el punto y coma (;). Este formato facilita el parseo línea por línea.

Para atributos que representan una colección de elementos simples, como la lista de amenidades de un alojamiento, los elementos dentro de ese campo se separan mediante un sub-delimitador, que es la

barra vertical ('—'). Por ejemplo, una lista de amenidades se guardaría como `Piscina|WiFi|Cocina`.

Las relaciones entre entidades se manejan almacenando los identificadores únicos de los objetos relacionados. Por ejemplo, un registro de alojamiento incluye el número de documento de su anfitrión responsable, y una reserva incluye el código del alojamiento y el documento del huésped. Las fechas se almacenan descomponiéndolas en sus componentes de día, mes y año, cada uno como un campo numérico separado. Los campos textuales, como nombres o anotaciones, se guardan directamente, tomando la precaución de que no contengan el delimitador principal para evitar conflictos durante el parseo.

Este diseño busca un equilibrio entre legibilidad humana de los archivos y la facilidad para el parseo y la reconstrucción de los objetos y sus relaciones en memoria durante las operaciones de carga y guardado del sistema.

6. PROBLEMAS DE DESARROLLO AFRONTADOS

Durante el desarrollo del proyecto UdeAStay, se presentaron diversos desafíos:

- **Gestión Manual de Memoria Dinámica:** La prohibición de usar STL exigió un manejo cuidadoso de la memoria para cadenas (`'char*'`) y para la implementación de `'ListaPersonalizada'`, siendo crucial la correcta aplicación de la Regla de Tres/Cinco.
- **Implementación de Estructuras de Datos Propias:** Crear `'ListaPersonalizada;Tz'` de forma genérica y eficiente fue un reto.
- **Parseo de Archivos de Texto:** Sin las utilidades de `'std::string'` o `'stringstream'`, la tokenización de líneas fue laboriosa.
- **Enlace de Objetos y Consistencia:** Reconstruir relaciones y mantener la consistencia de datos durante la carga desde archivos requirió un diseño cuidadoso.
- **Aritmética de Fechas:** Implementar cálculos de fechas precisos sin bibliotecas estándar fue complejo.
- **Errores de Compilación y Lógica:** La depuración de errores relacionados con punteros, `'const'`-correctness y la lógica de las funcionalidades fue un proceso iterativo.

7. EVOLUCIÓN DE LA SOLUCIÓN Y CONSIDERACIONES PARA IMPLEMENTACIÓN

7.1. Evolución de la Solución

El proyecto evolucionó desde un análisis inicial y un diseño de clases preliminar (que consideró herencia para roles de usuario), pasando por un refinamiento que optó por la composición, hasta la implementación iterativa de clases auxiliares y principales. El

diseño de la persistencia y la gestión del ciclo de vida de las reservas fueron puntos clave ajustados durante el desarrollo.

7.2. Consideraciones para Tener en Cuenta en la Implementación (y Futuro)

- **Pruebas Exhaustivas:** Es fundamental validar la E/S de archivos, la gestión de memoria, la lógica de fechas y todas las funcionalidades.
- **Manejo de Errores Robusto:** Mejorar la gestión de errores en entradas de usuario y lectura de archivos.
- **Clase 'MiString' Propia:** Podría simplificar el manejo de 'char*' y reducir errores.
- **Eficiencia:** Para grandes volúmenes de datos, la búsqueda en 'ListaPersonalizada' podría optimizarse.
- **Seguridad de Credenciales:** El almacenamiento de contraseñas en texto plano es una simplificación; en un entorno real, se usaría hashing.
- **Documentación del Formato de Archivo:** Detallar exhaustivamente el formato de los archivos de texto.

8. CONCLUSIONES

El desarrollo del proyecto UdeAStay ha permitido aplicar y consolidar los conocimientos de POO en C++ bajo restricciones desafiantes. Se ha logrado un sistema que modela las entidades y funcionalidades clave de un mercado de estadios hogareños, con persistencia de datos y manejo manual de memoria. Los desafíos afrontados han proporcionado un aprendizaje valioso. La solución actual constituye una base funcional que cumple con los requisitos fundamentales, y las consideraciones presentadas pueden guiar futuras mejoras.